TECHLEARN ACADEMY

Execução de algoritmos com matrizes

Curso

Lógica de Programação e Estruturas de Dados

Módulo 2 - Vetores, matrizes e execução de algoritmos

Como estudar esta matéria

1. Leia a explicação teórica e acompanhe os exemplos do material.

2. Resolva os exercícios comentados antes de conferir suas justificativas. Anote suas dúvidas para conversar com o mentor.

3. Ao terminar os materiais do módulo, faça a avaliação de revisão disponível no site.

Origem do conteúdo

CQA/UNIP - Tecnologia em Análise e Desenvolvimento de Sistemas, Tomo 3. Páginas 20 a 24 do arquivo original.

O conteúdo da matéria começa na página 4 deste PDF. Antes dele, foram mantidas a capa e a página de créditos e avisos do documento original.

Os textos e exercícios originais conservam suas referências e o contexto de publicação. A organização em cursos e as avaliações de revisão pertencem ao projeto TechLearn Academy.

UNIP
UNIVERSIDADE PAULISTA

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTITUCIONAL ESPECÍFICO

TOMO 3

CQA - COMISSÃO DE QUALIFICAÇÃO E AVALIAÇÃO

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

MATERIAL INSTRUCIONAL ESPECÍFICO

TOMO 3

Material instrucional específico, cujo conteúdo integral ou parcial não pode ser reproduzido ou utilizado sem autorização expressa, por escrito, da CQA/UNIP – Comissão de Qualificação e Avaliação da UNIP – UNIVERSIDADE PAULISTA.

Questão 6

Questão 6.[6]

Leia o algoritmo a seguir.

```
1   Algoritmo MatrizENADE2008
2   variaveis
3        M[0..2][0..3], I, J, C : inteiro
4   inicio
5        C ← 0
6        para I ← 0 ate 2 passo 1 faca
7        inicio
8        para J ← 0 ate 3 passo 1 faca
9        inicio
10            C ← C + 1
11            M[I][J] ← C
12       fim para
13  fim para
14  para I ← 0 ate 2 passo 1 faca
15  inicio
16       para J ← 0 até 3 passo 1 faca
17       inicio
18           C ← M[2-I][3-J]
19          M[I][J] ← C
20        fim para
21       fim para
22  fim algoritmo
```

Considerando a execução do algoritmo apresentada acima, faça o que se pede a seguir.

A. Apresente os dados da matriz M ao término da execução da linha 12.

B. Apresente os dados da matriz M ao término da execução da linha 19.

1. Introdução teórica

Algoritmos

As questões envolvendo a interpretação de resultados de algoritmo demandam muita atenção e cuidado. O erro em um passo impossibilita encontrar o resultado correto para toda a questão.

É fundamental entender o que o algoritmo está fazendo. Ele executa as seguintes macroatividades:

- inicializa as variáveis (linha 3);
- realiza o primeiro *loop* (laço ou execução) na matriz bidimensional (linhas 5 a 13);
- realiza o segundo *loop* na matriz bidimensional (linhas 14 a 21).

[6]Questão 40 - Discursiva – Enade 2008.

O ponto a ser observado é que a questão não solicita a verificação dos resultados ao final de cada *loop*, e sim em pontos intermediários (linhas 12 e 19). Se tivesse solicitado que fossem mostrados os resultados após as linhas 13 e 21, então poderíamos esperar a completa execução dos dois *loops* e depois mostrar o resultado. Mas não é o caso, o que demanda cuidado adicional.

Um ponto importante é que as variáveis numéricas são inicializadas com zero. Portanto, a matriz bidimensional, após a linha 3, tem o seguinte formato:

0 0 0 0
0 0 0 0
0 0 0 0

É uma matriz 3x4, com os índices variando de 0 a 2 (3 valores) e de 0 a 3 (4 valores). O total de membros é 12. A matriz mostrada acima é a entrada do algoritmo. Ela sofrerá o processamento, conforme será explicado a seguir.

2. Indicações bibliográficas

- CORMEN, T. H. *Algoritmos - teoria e prática.* São Paulo: Campus, 2002.
- DASGUPTA, S.; PAPADIMITRIOU, C. H.; VAZIRANI, U. *Algoritmos.* São Paulo: McGraw-Hill, 2006.
- GOODRICH, M. T.; TAMASSIA, R. *Projeto de algoritmo.* Porto Alegre: Bookman, 2004.
- MEDINA, M.; FERTIG, C. *Algoritmos e programação - teoria e prática.* São Paulo: Novatec, 2005.

3. Resolução da questão

A. A entrada para o processamento é a matriz bidimensional com todos os valores zerados, conforme abaixo:

0 0 0 0
0 0 0 0
0 0 0 0

É fundamental entender como a matriz bidimensional está sendo processada na questão. O índice I varre as linhas e o índice J varre as colunas. Então, para o índice I começando com 0 (zero), o índice J variará de 0 (zero) até 3 (três). Como a questão solicita a visualização da matriz após a linha 12, o índice I somente conterá o valor 0 (zero). Isso

significa que teremos de tirar uma foto da matriz no momento em que o índice I for zero e o índice J variar de 0 até 3.

O quadro 1 mostra a evolução do processamento até a linha 12.

Quadro 1. Evolução do processamento até a linha 12.

I	J	C	M[I,J]
0	0	1	1
0	1	2	2
0	2	3	3
0	3	4	4

Portanto, a saída da matriz após a linha 12 é a seguinte:

1 2 3 4
0 0 0 0
0 0 0 0

C. Para resolver o segundo item da questão, é fundamental entender que a entrada para o item B não é a saída do item A. Ou seja, para que tenhamos os dados corretos que iniciarão o segundo *loop*, não podemos considerá-los como sendo a resposta ao item A, que solicitou a visualização após a linha 12. A entrada para o item B é a matriz resultante após a linha 13. Portanto, teremos que continuar o processamento do algoritmo após a linha 12, conforme apresentado no quadro 2.

D.

Quadro 2. Evolução do processamento até a linha 13.

I	J	C	M[I,J]
0	0	1	1
0	1	2	2
0	2	3	3
0	3	4	4
1	0	5	5
1	1	6	6
1	2	7	7
1	3	8	8
2	0	9	9
2	1	10	10
2	2	11	11
2	3	12	12

Portanto, a matriz que é a entrada para o processamento do item B é:

1 2 3 4
5 6 7 8
9 10 11 12

Outro ponto de atenção é o momento em que é pedida a visualização da matriz do item B. Na linha 19, diferentemente do item A, só é processada uma coluna do segundo *loop*, a posição onde I=0 e J=0. A atribuição é feita trazendo o dado que se encontra na posição M[2-I][3-J] para a posição M[I][J].

Como tanto I quanto J valem 0 (zero) nesse momento do processamento, a posição é M[2][3], porque 2-0=0 e 3-0=3. Essa é a posição de origem. A posição de destino é M[0][0].

O valor que se encontra em M[2][3] é 12. Esse valor será colocado em M[0][0]. Com isso, a matriz fica conforme abaixo:

12 2 3 4
5 6 7 8
9 10 11 **12**

O programa Java abaixo pode ser usado para verificar os resultados apresentados.

```
package enade;
public class Main {
    public static void main(String[] args) {
        int M[][] = new int[3][4];
        int c = 0;
        for (int I = 0; I < 3; I++) {
            for (int J = 0; J < 4; J++) {
                c = c + 1;
                M[I][J] = c;
            }
        }
        for (int I = 0; I < 3; I++) {
            for (int J = 0; J < 4; J++) {
                c = M[2 - I][3 - J];
                M[I][J] = c;
            }
        }
    }
}
```

Para ver o resultado do item A, execute o programa abaixo, que inclui a visualização da matriz no ponto exato após a linha 12.

```
package enade;
public class Main {
    public static void main(String[] args) {
        int M[][] = new int[3][4];
        int c = 0;
        for (int I = 0; I < 3; I++) {
            for (int J = 0; J < 4; J++) {
                c = c + 1;
                M[I][J] = c;
            }
            System.out.println(M[0][0] + " " + M[0][1] + " " + M[0][2] + " " +
M[0][3]);
            System.out.println(M[1][0] + " " + M[1][1] + " " + M[1][2] + " " +
M[1][3]);
            System.out.println(M[2][0] + " " + M[2][1] + " " + M[2][2] + " " +
M[2][3]);
            System.out.println(" ");
            return;
```

```
        }
        for (int I = 0; I < 3; I++) {
            for (int J = 0; J < 4; J++) {
                c = M[2 - I][3 - J];
                M[I][J] = c;
            }
        }
    }
}
```

Para ver o resultado do item B, execute o programa a seguir, que inclui a visualização da matriz no ponto exato após a linha 19.

```
package enade;
public class Main {
    public static void main(String[] args) {
        int M[][] = new int[3][4];
        int c = 0;
        for (int I = 0; I < 3; I++) {
            for (int J = 0; J < 4; J++) {
                c = c + 1;
                M[I][J] = c;
            }
        }
        for (int I = 0; I < 3; I++) {
            for (int J = 0; J < 4; J++) {
                c = M[2 - I][3 - J];
                M[I][J] = c;
                System.out.println(M[0][0] + " " + M[0][1] + " " + M[0][2] + "
" + M[0][3]);
                System.out.println(M[1][0] + " " + M[1][1] + " " + M[1][2] + "
" + M[1][3]);
                System.out.println(M[2][0] + " " + M[2][1] + " " + M[2][2] + "
" + M[2][3]);
                System.out.println(" ");
                return;
            }
        }
    }
}
```